Our World in Data Energy

Dataset-Specific Laboratory Workbook

Data Acquisition & Manipulation with Pandas

GROUP Group B • Source: Our World in Data Energy dataset — CSV/XLSX/JSON + codebook

How to use this workbook

You support an energy-policy research unit. You must acquire a wide longitudinal dataset efficiently, select defensible indicators, and distinguish countries from aggregate regions.

This workbook assumes Python essentials and notebook use. Its focus is not Python syntax revision; it is disciplined Pandas reasoning: acquire → inspect → diagnose → select/filter → clean/transform → group/aggregate → merge/reconcile → export.

ROW GRAIN CSV/XLSX: one row per location and year; many metrics are annual. The codebook documents indicators and sources.

KEYS & IDENTITY country + year is the natural location-year grain. iso_code helps distinguish many country rows; aggregate rows require deliberate handling.

Learning outcomes

- Acquire the dataset using the source-appropriate Pandas/API/database syntax.
- Diagnose structure, dtypes, missingness, duplicates, ranges, categories and memory implications before cleaning.
- Use loc/iloc, boolean masks, query, isin, between, sort_values and column selection intentionally.
- Clean and transform without overwriting raw data or silently changing analytical meaning.
- Use groupby, agg, named aggregation, value_counts, pivot/crosstab/transform where justified.
- Join/merge/concat with explicit key/cardinality reasoning and post-operation validation.
- Export reproducible outputs and explain why each transformation was necessary.

Acquisition starter

```
import pandas as pd
url = "https://owid-public.owid.io/data/energy/owid-energy-data.csv"

# First inspect header only
cols = pd.read_csv(url, nrows=0).columns.tolist()
print(cols[:30])

wanted = ["country","year","iso_code","population",
          "primary_energy_consumption","electricity_demand",
          "renewables_consumption","fossil_fuel_consumption"]
wanted = [c for c in wanted if c in cols]
energy = pd.read_csv(url, usecols=wanted)
```

```
# Alternative Excel
# energy_xlsx = pd.read_excel("owid-energy-data.xlsx")

# Codebook
codebook = pd.read_csv("owid-energy-codebook.csv")
```

Core Pandas syntax map

Capability	Methods / properties
Inspect	shape, head(), tail(), sample(), columns, dtypes, info(), memory_usage(), describe(), nunique(), value_counts(), isna(), duplicated()
Select	df[col], df[[...]], loc[rows, cols], iloc[rows, cols], filter(), select_dtypes()
Filter	boolean masks with & | ~, isin(), between(), str methods, query(), notna()/isna()
Clean	copy(), dropna(), fillna(), drop_duplicates(), astype(), to_numeric(), to_datetime(), str.strip(), replace(), map(), rename()
Transform	assign(), where(), mask(), cut(), qcut(), apply()/map() when justified; prefer vectorization
Group	groupby(), size(), count(), sum(), mean(), median(), min(), max(), nunique(), agg(), transform(), pivot_table(), crosstab()
Combine	merge(how=..., on=..., validate=..., indicator=True), join(), concat(), suffixes
Validate/export	assert, equals(), compare counts/totals, to_csv(index=False), to_excel(index=False), to_parquet()

Case Study 1: Efficient acquisition of a wide dataset

REAL-LIFE QUESTION Why inspect headers before loading 100+ columns, and how much memory does usecols save?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

1. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
2. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
3. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
4. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
5. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
6. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
energy.shape
energy.info()
energy.memory_usage(deep=True).sum()/1024**2
energy[["country","year"]].duplicated().sum()
energy.isna().mean().sort_values(ascending=False).head(20)
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Case Study 2: Filter/select countries and periods

REAL-LIFE QUESTION How do you avoid accidentally mixing world/region aggregates with countries?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

7. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
8. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
9. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
10. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
11. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
12. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
country_rows = energy.loc[energy["iso_code"].notna()].copy()
recent = country_rows.loc[
    country_rows["year"].between(2015, 2024),
    ["country","iso_code","year"] + [c for c in wanted if c not in {"country","iso_code","year"}]
].copy()
recent.sort_values(["country","year"]).head()
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?

- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Case Study 3: Cleaning structural missingness

REAL-LIFE QUESTION Why is filling every missing energy indicator with zero analytically dangerous?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

13. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
14. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
15. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
16. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
17. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
18. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
e = recent.copy()
# Diagnose missingness by variable and country; do not blanket fill.
missing = e.isna().mean().sort_values(ascending=False)
print(missing)


# Example feature only where inputs exist
if {"renewables_consumption","fossil_fuel_consumption"}.issubset(e.columns):
    denom = e["renewables_consumption"] + e["fossil_fuel_consumption"]
    e["renewable_share_selected"] = e["renewables_consumption"].div(denom).where(denom.gt(0))
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?

- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Case Study 4: Group & aggregate

REAL-LIFE QUESTION What is the latest available observation per country, and why can groupby.last() be misleading with nulls?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

19. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
20. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
21. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
22. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
23. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
24. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
latest_year = e.groupby("country")["year"].max().rename("latest_year")
latest = e.merge(latest_year, on="country", how="left", validate="many_to_one")
latest = latest.loc[latest["year"].eq(latest["latest_year"])].copy()

# Period summary
metrics=[c for c in ["primary_energy_consumption","electricity_demand"] if c in e.columns]
period = e.groupby("country")[metrics].agg(["count","mean","min","max"])
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Case Study 5: Merge with codebook / external indicators

REAL-LIFE QUESTION Can you reshape selected indicators to long form and attach definitions from the codebook?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

25. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
26. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
27. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
28. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
29. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
30. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
value_cols=[c for c in ["primary_energy_consumption","electricity_demand"] if c in e.columns]
long = e.melt(id_vars=["country","iso_code","year"],
              value_vars=value_cols,
              var_name="column", value_name="value")

# Codebook key is 'column' in the repository's legacy codebook.
cb = codebook.drop_duplicates("column")
long_meta = long.merge(cb, on="column", how="left",
                       validate="many_to_one", indicator=True)
long_meta["_merge"].value_counts()
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Final Integrated Case

Management gives you the raw source and asks for a decision-ready table. Build an end-to-end notebook without copying the guided solution blocks. Your notebook must contain:

- source/provenance note and acquisition code
- grain and key statement
- dataset-health table
- documented filtering population
- cleaning decision log
- at least two derived features where meaningful
- at least two grouped summaries
- one merge/concat/reconciliation task appropriate to this dataset
- three validation assertions/checks
- clean export with index=False where CSV/Excel is used
- 200–300 word management note stating findings, exclusions, assumptions and limitations

Common pitfalls — non-negotiable

Pitfall	Required response
and / or in Series filters	Use & / | with each condition in parentheses.
Chained assignment	Use .loc[...] = ...; do not use df[df['a']>5]['b']=0.
Comparing NaN with ==	Use .isna() / .notna().
Modifying a slice	Use .copy() when creating a working subset.
Ignoring dtypes	Run info() early; IDs often should not be numeric measures.
Blind dropna/fillna	First explain what missing means in that field and question.
iterrows on large data	Prefer vectorized expressions/groupby/merge.
Unvalidated merge	Check key uniqueness, use validate= and indicator=True, reconcile rows/totals.
Overwriting raw files	Keep raw immutable; write clean/derived files separately.
CSV index leakage	Use to_csv(..., index=False) unless the index is intentionally part of the data.

Oral defence prompts

31. What does one row represent?
32. Which columns are identifiers rather than measures?
33. Which missing values did you retain, drop or impute—and why?
34. Show one operation that changed row count. Prove the change was intended.
35. What was the cardinality of your merge? How did you verify it?
36. Which metric would be wrong if you counted rows instead of unique business entities?
37. What would you change if this dataset had 50 million rows?
38. Which part belongs in SQL/API/source filtering rather than Pandas?
39. What conclusion can you NOT make from this dataset?

END OF DATASET LAB • Preserve raw data • Explain every decision • Validate every join